

Array in Java

17 July 2026 05:32

What is an Array?

Collection of items, having same data type, and they reside on the memory in the contiguous way.

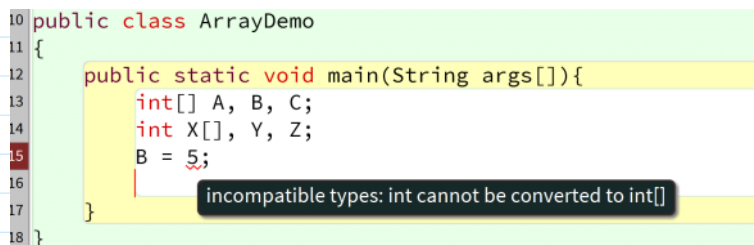
In Java when we allocate memory for the array it cannot be increased or decreased. The index of the array always starts from 0 and ends at N-1 for N elements.

Syntax :

```
dataType[] varivable_name;  
Or  
dataType varivable_name[];
```

For example:

```
public class ArrayDemo  
{  
    public static void main(String args[]){  
        int[] A, B, C; //Multiple Array Declaration  
        int X[], Y, Z; //Only X is Declared as an Array  
        //B = 5;  
        Y = 5;  
    }  
}
```



The screenshot shows the same code as above in an IDE. A red squiggly line under the assignment 'B = 5;' is highlighted with a tooltip that reads: 'incompatible types: int cannot be converted to int[]'. This indicates that the variable 'B' is declared as an array of integers, so it must be assigned an array of integers, not a single integer value.

Allocating memory for the array

The **new** keyword is used to allocate memory for the array.

Syntax:

```
Array_name = new dataType[N]; //N is number of elements.  
OR  
dataType array_name[] = new dataType[N];  
OR  
dataType[] array_name = new dataType[N];
```

For example:

```
public static void main(String args[]){  
    int[] A, B, C; //Multiple Array Declaration  
    int X[], Y, Z; //Only X is Declared as an Array  
    //B = 5;  
    Y = 5;  
    A = new int[5];  
    B = new int[3];
```

```

    int M[] = new int[10];
    int[] N = new int[20];
}

```

WAP to create an Array of size N, and store all the elements in an order.
For example:

5	10	12	15	18	20
---	----	----	----	----	----



Enter number of elements: 6
Enter any number: 10
Enter any number: 20
Enter any number: 15
Enter any number: 18
Enter any number: 5
Enter any number: 12

Output:

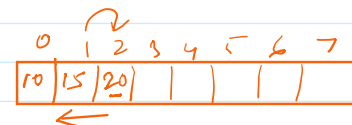
[5, 10, 12, 15, 18, 20]

→ → →

```

10 public class P1
11 {
12     public static void main()
13     {
14         Scanner sc=new Scanner (System.in);
15         System.out.println("\nEnter the size");
16         int n=sc.nextInt(),a[]=new int [n];
17         for(int i=0;i<n;i++)
18         {
19             System.out.println("Enter the array element");
20             int x=sc.nextInt(),j;
21             for(j=i-1;j>=0&& x<a[j];j--)
22             {
23                 a[j+1]=a[j];
24             }
25             a[j+1]=x;
26             System.out.println("Entered Array");
27             for(int i=0;i<n;i++)
28             {
29                 System.out.print(a[i]+" ");
30             }
31         }
32     }
33 }

```



$x = 10, 20, 15$
 $j = 1, 0, 0$
 $i = 0, 1, 2$

insertion at proper place.

basic of backtracking Algorithm.

Without sorting,

Write a Program to create an array of size N, and find the second highest value from it.

Homework:

1: Ordered Insertion in an Array (Without Sorting)

Write a program in Java to input N integers into an array in ascending order without using any sorting algorithm.

The user may enter the numbers in any order. As each number is entered, insert it into its correct position in the array by shifting the larger elements one position to the right.

Display the array after every insertion and finally print the complete sorted array.

Sample Input

Enter number of elements: 6
Enter number: 20
Enter number: 10
Enter number: 15

Enter number: 5
Enter number: 18
Enter number: 12

Output

After inserting 20

[20]

After inserting 10

[10, 20]

After inserting 15

[10, 15, 20]

After inserting 5

[5, 10, 15, 20]

After inserting 18

[5, 10, 15, 18, 20]

After inserting 12

[5, 10, 12, 15, 18, 20]

Restrictions

- Do not use `Arrays.sort()`.
- Do not use any Collection classes.
- Insert each element at its proper position while taking input.

2: Second Highest Distinct Element

Write a program to input N integers into an array and determine the second highest distinct element without sorting the array.

If the second highest distinct element does not exist, display an appropriate message.

Sample Input 1

8

15 22 11 45 45 10 39 22

Output

Highest = 45

Second Highest = 39

Sample Input 2

5

20 20 20 20 20

Output

Second Highest Element does not exist.

Restrictions

- Do not sort the array.
- Traverse the array only once after input.
- Do not use an additional array.

3: Insert an Element at a Given Position

Write a program to create an array of N elements.

Input a new element X and a position P (0-based index). Insert the element at the specified position by shifting the remaining elements to the right.

If the array is already full, display "Insertion not possible."

Example

Original Array

[10, 20, 30, 40, 50]

Enter element : 25

Enter position : 2

Output

Array after insertion

[10, 20, 25, 30, 40, 50]

Restrictions

- Do not create another array.
- Perform insertion using shifting only.

4: Maximum Difference

Write a program to input N integers into an array.

Find the largest positive difference between any two elements such that the larger element appears after the smaller element.

If no such pair exists, print 0.

Example

Input

9

7 1 5 3 6 4 8 2 10

Output

Largest Difference = 9

(10 - 1)

Example

Input

5

10 9 8 7 6

Output

Largest Difference = 0

Restrictions

- Do not sort the array.
- Use only one traversal after taking input.

5: Array Rotation Challenge

Write a program to input N integers into an array.

Input an integer K and rotate the array to the right by K positions.

If K is greater than N, rotate by the effective number of positions.

Example

Input

Array

10 20 30 40 50 60

K = 2

Output

50 60 10 20 30 40

Restrictions

- Do not use another array of size N.
- Perform the rotation manually.

6: Longest Consecutive Increasing Sequence

Write a program to input N integers into an array.

Determine the longest consecutive increasing sequence present in the array.

Print

- Starting index
- Ending index
- Length of the sequence
- Elements of the sequence

Example

Input

12

5 8 10 15 3 7 9 11 13 2 4 6

Output

Longest Consecutive Increasing Sequence

7 9 11 13

Starting Index = 5

Ending Index = 8

Length = 4

Example

Input

8

9 8 7 6 5 4 3 2

Output

Longest Consecutive Increasing Sequence

9

Length = 1

Restrictions

- Do not sort the array.
- Do not use any additional array.
- Solve using a single traversal.

7: Smallest Missing Positive Integer

Write a program in Java to input N integers into an array. The array may contain positive numbers, negative numbers, and zero.

Determine the smallest positive integer that is not present in the array.

Sample Input 1

Enter number of elements: 8

3 4 -1 1 7 2 6 8

Output

Smallest Missing Positive Integer = 5

Sample Input 2

Enter number of elements: 5

1 2 3 4 5

Output

Smallest Missing Positive Integer = 6

Restrictions

- Do not sort the array.
- Do not use any Collection classes (ArrayList, HashSet, etc.).
- Do not use library search methods.
- Solve the problem using array traversal and logical reasoning.

8: Leaders in an Array

An element in an array is called a Leader if it is greater than every element to its right. The last element of the array is always considered a leader.

Write a Java program to input N integers into an array and display all the leader elements in the order in which they appear.

Sample Input

Enter number of elements: 8

16 17 4 3 5 2 8 1

Output

Leader Elements are:

17

8

1

Sample Input

Enter number of elements: 6

30 25 20 15 10 5

Output

Leader Elements are:

30
25
20
15
10
5

Restrictions

- Do not sort the array.
- Do not use another array to store the leaders.
- Display the leaders in their original order of appearance.
- Aim to solve the problem with a single traversal from right to left.

What are Multi-Dimensional Arrays in Java?

Multi-dimensional array is array of arrays, where each element of the main array holds another array. They are commonly used to represent data in table or matrix form, such as grids, charts or spreadsheets.

Multi-dimensional array can be compared to a table. For example storing marks of students where each row represents the student and each column represents a subject. In this way data is organized in rows and columns using a multi-dimensional array.

Two-dimensional (2D) Array

A two dimensional array (2D) array is a type of multi - dimensional array that stores data in the form of rows and columns. It is used to represent table - like structures such as matrices, grids, or tables, where data is organized in two dimensional only.

Declaration

You can use one of the following syntaxes to declare a 2 dimensional array.

Syntax:

Declaration:

```
DataType[][] arrayName;  
Or  
DataType arrayName[][];
```

For memory allocation:

```
arrayName = new dataType[numberOfRows][NumberOfColumns];
```

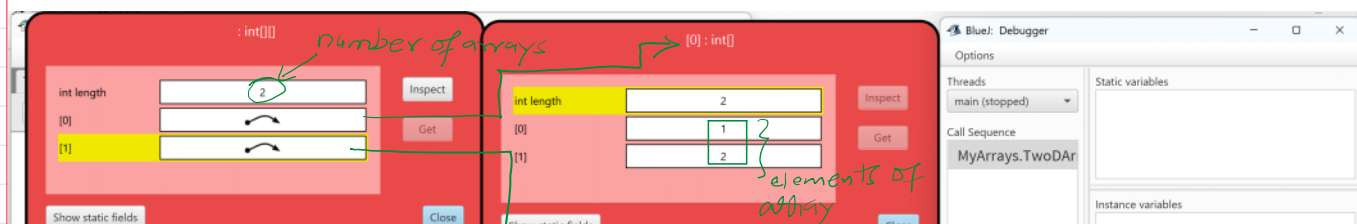
OR

Declaration along with memory allocation:

```
DataType[][] arrayName = new dataType[numberOfRows][NumberOfColumns];
```

OR

```
DataType arrayName[][] = new dataType[numberOfRows][NumberOfColumns];
```



elements of array

```

7  * @author (your name)
8  * @version (a version number or a date)
9  */
10 public class TwoDArrayOper
11 {
12     public static void main(String[] args){
13         int[][] arr = {{1,2}, {3,4}};
14     }
15 }

```

int length: 2

[0]	3
[1]	4

String[] args = <object reference>
int[][] arr = <object reference>

int length: 3

[0]	0
[1]	0
[2]	0

default values assigned by Java compiler

```

10 public class TwoDArrayOper
11 {
12     public static void main(String[] args){
13         int[][] arr = {{1,2}, {3,4}};
14         //Default value of elements in an array
15         int[][] arr1 = new int[3][3];
16     }
17 }

```

memory of 2D array is allocated by 'new' keyword dynamically during the execution of code, on the heap memory area.

int length: 3

[0]	1
[1]	2
[2]	3
[3]	4

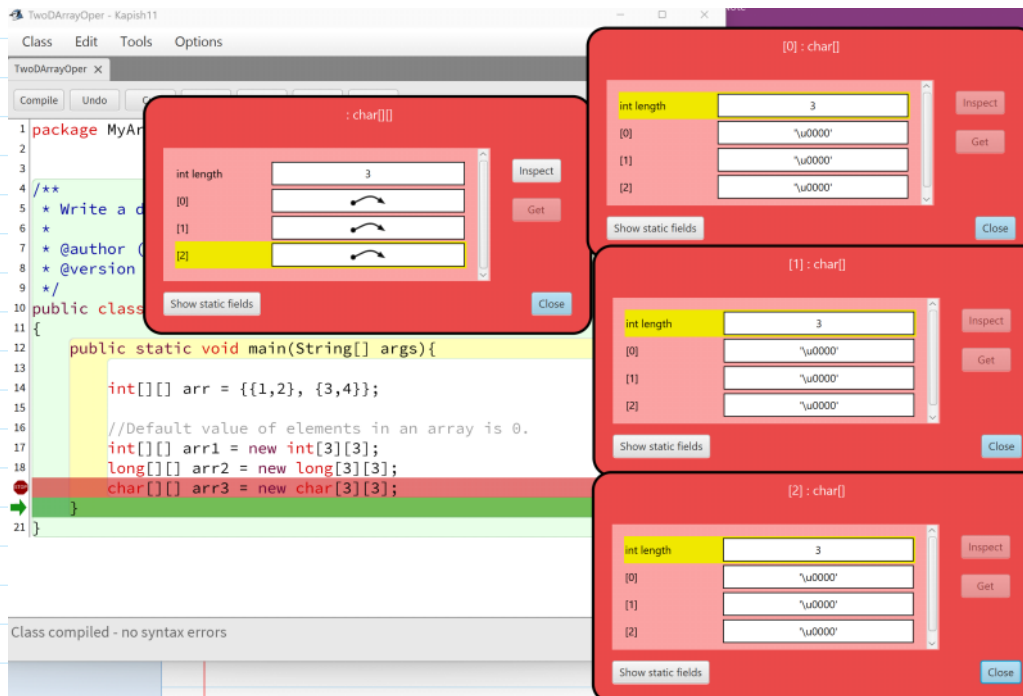
Default values assigned by Java compiler in array:

For integer type array: 0 (byte, short, int, long)

For floating point type: 0.0

For boolean: false

For char : '\u0000' or null



The representation of 2D array

The following email shows the representation of 2D array:

rows

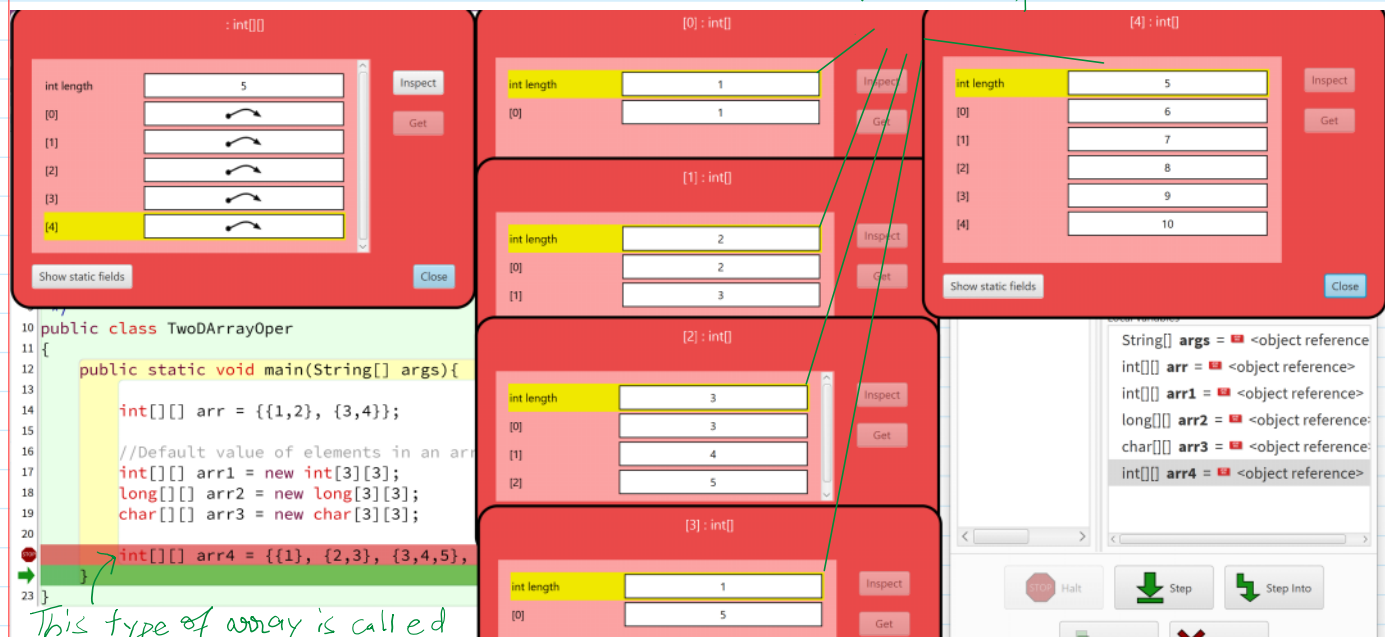
	0	1	2
0	0,0	0,1	0,2
1	1,0	1,1	1,2
2	2,0	2,1	2,2

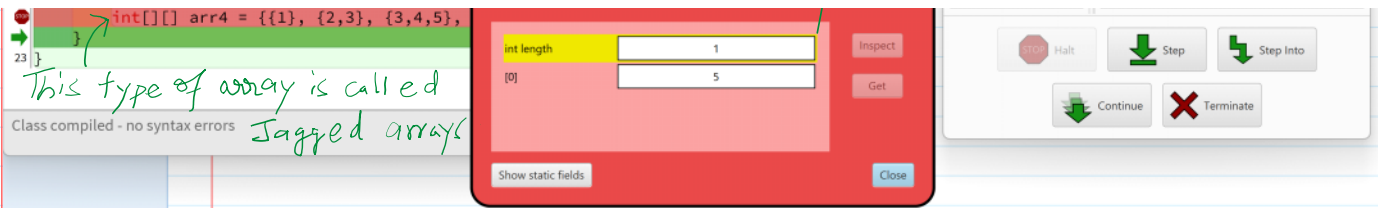
columns

Two dimensional array in memory:

0,0 0,1 0,2 1,0 1,1 1,2 2,0 2,1 2,2

number of elements in each row varies





```
package MyArrays;
```

```
/**
```

```
 * Write a description of class TwoDArrayOper here.
```

```
 *
```

```
 * @author (your name)
```

```
 * @version (a version number or a date)
```

```
 */
```

```
public class TwoDArrayOper
```

```
{
```

```
    public static void main(String[] args){
```

```
        int[][] arr = {{1,2}, {3,4}};
```

```
        //Default value of elements in an array is 0.
```

```
        int[][] arr1 = new int[3][3];
```

```
        long[][] arr2 = new long[3][3];
```

```
        char[][] arr3 = new char[3][3];
```

```
        int[][] arr4 = {{1}, {2,3}, {3,4,5}, {5},{6,7,8,9,10}};
```

```
        int i, j;
```

```
        System.out.print("\fTechnique-1\n");
```

```
        for(i = 0; i < arr4.length; i++, System.out.println()){
```

```
            for(j = 0; j < arr4[i].length; j++){
```

```
                System.out.print("\t"+arr4[i][j]);
```

```
            }
```

```
        }
```

```
        System.out.print("\nTechnique-2\n");
```

```
        //for each loop only applicalable for collection(Array, list,...)
```

```
        for(int[] x: arr4){
```

```
            for(i = 0; i < x.length; i++){
```

```
                System.out.print("\t"+x[i]);
```

```
            }
```

```
            System.out.println();
```

```
        }
```

```
        System.out.print("\nTechnique-3\n");
```

```
        //for each loop only applicalable for collection(Array, list,...)
```

```
        for(int[] x: arr4){
```

```
            System.out.println(java.util.Arrays.toString(x));
```

```
        }
```

```
        System.out.print("\nTechnique-4\n");
```

```
        //for each loop only applicalable for collection(Array, list,...)
```

```
        for(i = 0; i < arr4.length; i++){
```

```

        System.out.println(java.util.Arrays.toString(arr4[i]));
    }
}

```

Three Dimensional (3D) Array

A 3 dimensional (3D) array is an array that contains arrays of arrays. It is used to store data in three dimensions such as layers, rows and columns. A 3D array is commonly used to represent complex data structures like a collection of matrices or data arranged across multiple levels.

It is a complex form of 2D array. In other words it is an array of two dimensional array.

Declaration

Syntax:

Declaration:

```
DataType[][][] arrayName;
```

Or

```
DataType arrayName[][][];
```

For memory allocation:

```
arrayName = new dataType[layers][numberOfRows][NumberOfColumns];
```

OR

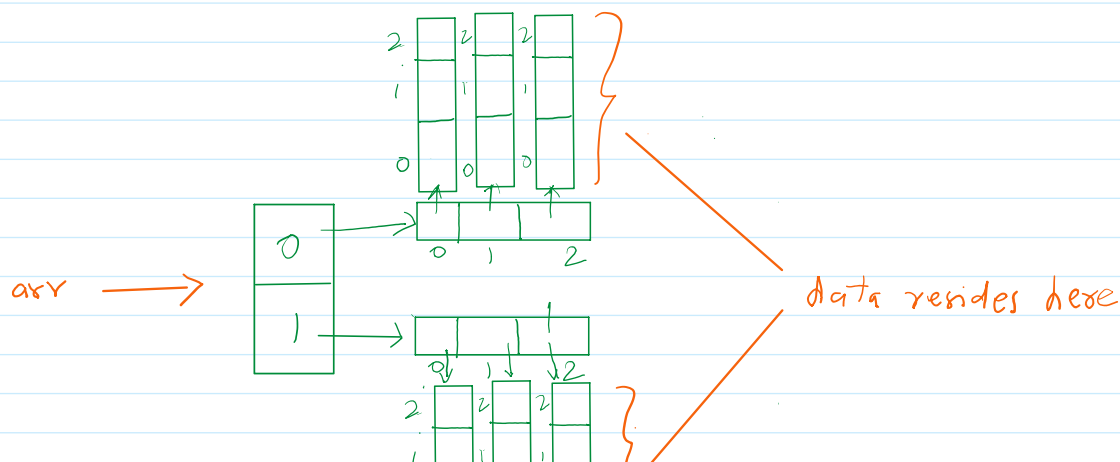
Declaration along with memory allocation:

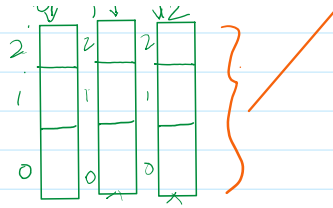
```
DataType[][][] arrayName = new dataType[layers][numberOfRows]
[NumberOfColumns];
```

OR

```
DataType arrayName[][][] = new dataType[layers][numberOfRows]
[NumberOfColumns];
```

```
int[][][] arr = new int[2][3][3];
```





Three dimensional Array

```
package MyArrays;
import java.util.*;

/**
 * Write a description of class ThreeDArray here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ThreeDArray
{
    public static void main(String[] args){
        int[][][] arr = new int[2][2][3];
        int i, j, k;
        Random r = new Random();
        for(i = 0; i < arr.length; i++){
            System.out.print("\nLayer-"+i+"\n");
            for(j = 0; j < arr[i].length; j++, System.out.print("\n")){
                for(k = 0; k < arr[i][j].length; k++){
                    arr[i][j][k] = r.nextInt(10, 100);
                    System.out.print("\t"+arr[i][j][k]);
                }
            }
        }
    }
}
```

Output:

BlueJ: Terminal Window - Kapish11			
Options			
Layer-0			
97	53	85	
34	40	72	
Layer-1			
63	92	24	
98	72	30	

Notes In Java array is the template of Arrays class present in java.util. package. Therefore, internally it assigns number of elements in its length variable, which is a public instance variable in Arrays class.

array.length

↑ directly accessible.
because it is a public
int variable.